

21. Find the Union of Two Arrays

Description

Given two arrays, find all distinct elements present in either array.

Input Format

First line: N

Second line: N integers

Third line: M

Fourth line: M integers

Sample Input

5

1 2 3 4 5

4

1 2 6 7

Sample Output

1 2 3 4 5 6 7

Explanation

Union contains all unique elements from both arrays.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,m;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```

    }
    scanf("%d",&m);
    int arr2[m];
    for(int i=0;i<m;i++){
        scanf("%d",&arr2[i]);
    }
    for(int i=0;i<n;i++){
        printf("%d ",arr[i]);
    }
    int k=0;
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            if(arr2[i]==arr[j]){
                k++;
            }
        }
        if(k==0){
            printf("%d ",arr2[i]);
        }
        k=0;
    }

    return 0;
}

```

22. Find the Intersection of Two Arrays

Description

Find common elements present in both arrays.

Sample Input

5

1 2 3 4 5

5

3 4 5 6 7

Sample Output

3 4 5

Explanation

3, 4, and 5 are present in both arrays.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,m;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    scanf("%d",&m);
```

```
    int arr2[m];
```

```
    for(int i=0;i<m;i++){
```

```
        scanf("%d",&arr2[i]);
```

```
    }
```

```
    int k=0;
```

```
    for(int i=0;i<m;i++){
```

```
        for(int j=0;j<n;j++){
```

```
            if(arr2[i]==arr[j]){
```

```
                k++;
```

```

    }
    if(k>0){
        printf("%d ",arr2[i]);
    }
    k=0;
}

return 0;
}

```

23. Find the First Repeating Element

Description

Find the first element that appears more than once.

Sample Input

7

10 5 3 4 3 5 6

Sample Output

5

Explanation

5 is the first element whose repetition occurs earliest.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```

int arr[n];

for(int i=0;i<n;i++){
    scanf("%d",&arr[i]);
}

int i=0,j=1;
while(i<j){
    if(j<n && arr[i]==arr[j]){
        printf("First Repeating Element %d",arr[i]);
        return 0;
    }j++;
    if(j==n-1){
        i++;
        j=i+1;
    }
}

return 0;
}

```

24. Find the First Non-Repeating Element

Description

Find the first element that occurs exactly once.

Sample Input

6

9 4 9 6 7 4

Sample Output

6

Explanation

6 is the first element with frequency 1.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    int copy[10000];
```

```
    for(int i=0;i<n;i++){
```

```
        copy[arr[i]]=0;
```

```
    }
```

```
    for(int i=0;i<n;i++){
```

```
        copy[arr[i]]++;
```

```
    }
```

```
    for(int i=0;i<n;i++){
```

```
        if(copy[arr[i]]==1){
```

```
            printf("%d ",arr[i]);
```

```
            break;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

25. Maximum Consecutive Ones

Description

Find the maximum number of consecutive 1s in a binary array.

Sample Input

8

1 1 0 1 1 1 0 1

Sample Output

3

Explanation

The longest sequence of consecutive 1s has length 3.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int s=0;s<n;s++){
```

```
        scanf("%d",&arr[s]);
```

```
    }
```

```
    int max=0,c=0;
```

```
    for(int i=0;i<n;i++){
```

```
        if(arr[i]==1){
```

```
            c++;
```

```
        }
```

```
        if(c>max){
```

```

        max=c;
    }
    if(arr[i]==0){
        c=0;
    }
}printf("number of consecutive 1s in a binary array %d",max);

return 0;
}

```

26. Find All Duplicate Elements

Description

Print all elements that appear more than once.

Sample Input

8

1 2 3 1 2 4 5 2

Sample Output

1 2

Explanation

Elements 1 and 2 occur multiple times.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```



```

scanf("%d",&n);

int arr[n];

for(int i=0;i<n;i++){
    scanf("%d",&arr[i]);
}

int k=0;

for(int i=0;i<n;i++){
    for(int j=i+1;j<n;j++){
        if(arr[i]==arr[j]){
            k++;
        }
    }
    if(k>0){
        printf("%d ",arr[i]);
        k=0;
    }
}

return 0;
}

```

27. Largest Sum Contiguous Subarray of Size K

Description

Find the maximum sum among all subarrays of size K.

Sample Input

6

2 1 5 1 3 2

3

Sample Output

9

Explanation

Subarray [5,1,3] has sum 9.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int n,k,sum=0,maxsum=0;
    scanf("%d",&n);
    int arr[n];
    for(int s=0;s<n;s++){
        scanf("%d",&arr[s]);
    }
    scanf("%d",&k);
    for(int i=0;i<k;i++){
        sum+=arr[i];
    }
    maxsum=sum;
    int y=k;
    for(int i=k;i<n;i++){
        int x=(maxsum-arr[i-k])+arr[y];
        y++;
        maxsum=x;
        if(x>sum){
            sum=x;
        }
    }
    printf("largest sum contiguous subarray of k is %d",sum);

    return 0;
```

```
}
```

28. Find the Smallest Positive Missing Number

Description

Find the smallest positive integer missing from the array.

Sample Input

5

3 4 -1 1 2

Sample Output

5

Explanation

Numbers 1,2,3,4 exist, so 5 is missing.

PROGRAM:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
int comp(const void *a,const void *b){
```

```
    return(*(int*)a - *(int*)b);
```

```
}
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }int min=0,max=0;
```

```
    for(int i=0;i<n;i++){
```

```
        if(arr[i]>0){
```

```

        min=arr[i];
        max=arr[i];
        break;
    }
}
for(int i=0;i<n;i++){
    if(arr[i]>0 && min>arr[i]){
        min=arr[i];
    }
}
qsort(arr,n,sizeof(int),comp);
for(int i=0;i<n;i++){
    if(arr[i]>0 && arr[i]==min){
        min++;
    }
}
printf("%d",min);

return 0;
}

```

29. Count Inversions in an Array

Description

An inversion is a pair (i,j) such that $i < j$ and $arr[i] > arr[j]$.

Sample Input

5

2 4 1 3 5

Sample Output

3

Explanation

Inversions are (2,1), (4,1), and (4,3).

PROGRAM:

```
#include <stdio.h>
```

```
int main()
{
    int n,k=0;
    scanf("%d",&n);
    int arr[n];
    for(int s=0;s<n;s++){
        scanf("%d",&arr[s]);
    }
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(arr[i]>arr[j]){
                k++;
            }
        }
    }
    printf("%d",k);

    return 0;
}
```

30. Find Peak Element

Description

Find an element greater than its adjacent elements.

Sample Input

6

1 3 20 4 1 0

Sample Output

20

Explanation

20 is greater than both 3 and 4.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    for(int i=1;i<n-1;i++){
```

```
        if(arr[i]>arr[i-1] && arr[i]>arr[i+1]){
```

```
            printf("%d",arr[i]);
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

31. Sliding Window Maximum

Description

Find the maximum element in every window of size K.

Sample Input

8

1 3 -1 -3 5 3 6 7

3

Sample Output

3 3 5 5 6 7

Explanation

Each value is the maximum of a window of size 3.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,k;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int s=0;s<n;s++){
```

```
        scanf("%d",&arr[s]);
```

```
    }
```

```
    scanf("%d",&k);
```

```
    int y=0,i;
```

```
    while(y<=n-k){
```

```
        int max=arr[y];
```

```
        for(i=y;i<y+k;i++){
```

```
            if(max<arr[i]){
```

```
                max=arr[i];
```

```
        }
```

```
    }printf("%d ",max);  
    y++;  
  
}  
    return 0;  
}
```

32. Minimum Number of Jumps to Reach End

Description

Each element represents the maximum jump length from that position.

Sample Input

11

1 3 5 8 9 2 6 7 6 8 9

Sample Output

3

Explanation

Minimum jumps required to reach the last index are 3.

PROGRAM:

```
#include <stdio.h>
```

```
int main()  
{  
    int n,k;  
    scanf("%d",&n);  
    int arr[n];
```



```

for(int s=0;s<n;s++){
    scanf("%d",&arr[s]);
}int sum=0,i=0;
while(i<n){
    i+=arr[i];
    sum++;
}
printf("%d ",sum);
return 0;
}

```

33. Find Subarray with Given Sum

Description

Find the starting and ending indices of a subarray whose sum equals K.

Sample Input

5

1 4 20 3 10

33

Sample Output

2 4

Explanation

$20 + 3 + 10 = 33$.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,k;
```

```

scanf("%d",&n);
int arr[n];
for(int i=0;i<n;i++){
    scanf("%d",&arr[i]);
}
scanf("%d",&k);
for(int i=0;i<n;i++){
    int sum=0;
    for(int j=i;j<n;j++){
        sum+=arr[j];
        if(sum==k){
            printf("%d %d",i,j);
            return 0;
        }
    }
}
return 0;
}

```

34. Find the Median of Two Sorted Arrays

Description

Find the median of two sorted arrays.

Sample Input

3

1 3 5

3

2 4 6

Sample Output

3.5

Explanation

Merged array = [1,2,3,4,5,6].

Median = $(3+4)/2 = 3.5$.

PROGARM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,m;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    scanf("%d",&m);
```

```
    int arr2[m];
```

```
    for(int i=0;i<m;i++){
```

```
        scanf("%d",&arr2[i]);
```

```
    }
```

```
    int a[n+m];
```

```
    int k=0;
```

```
    for(int i=0;i<n;i++){
```

```
        a[k]=arr[i];
```

```
        k++;
```

```
    }
```

```
    for(int i=0;i<m;i++){
```

```
        a[k]=arr2[i];
```

```
        k++;
```

```
    }
```

```

for(int i=0;i<n+m;i++){
    for(int j=i+1;j<n+m;j++){
        if(a[i]>a[j]){
            int temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }
}

int size=n+m;
if(size%2==0){
    float median=(a[size/2-1]+a[size/2])/2.0;
    printf("%.1f",median);
}
else{
    printf("%d",a[size/2]);
}

return 0;
}

```

35. Maximum Length Bitonic Subarray

Description

Find the longest subarray that first increases and then decreases.

Sample Input

8

12 4 78 90 45 23 10 5

Sample Output

7

Explanation

Bitonic subarray is [4,78,90,45,23,10,5].

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    int max=1;
```

```
    for(int i=0;i<n;i++){
```

```
        int c=1;
```

```
        int j=i;
```

```
        while(j>0 && arr[j]>arr[j-1]){
```

```
            c++;
```

```
            j--;
```

```
        }
```

```
        j=i;
```

```
        while(j<n-1 && arr[j]>arr[j+1]){
```

```
            c++;
```

```
            j++;
```

```
        }
```

```
        if(c>max){
```

```
            max=c;
```

```
        }
```

```
    }
```

```
    printf("%d",max);
```

```
    return 0;
```

```
}
```

```
.
```

36. Find the K-th Largest Element

Description

Find the K-th largest element in the array.

Sample Input

6

7 10 4 3 20 15

3

Sample Output

10

Explanation

Sorted descending: 20,15,10,7,4,3.

3rd largest element is 10.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,k;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    scanf("%d",&k);
```

```
for(int i=0;i<n;i++){  
    for(int j=i+1;j<n;j++){  
        if(arr[i]<arr[j]){  
            int temp=arr[i];  
            arr[i]=arr[j];  
            arr[j]=temp;  
        }  
    }  
}  
printf("%d",arr[k-1]);  
return 0;  
}
```